02616 Large Scale Modelling - Assignment 1

s224363, s224408, s224059, s201725

October 18, 2026

1 Introduction to the problem and MPI

In this report we parallelize the computation of the Mandelbrot set using the Message Passing Interface (MPI) through its Python binding, mpi4py. MPI is a tool for communication between independent processes, which may run on the same machine, on separate nodes of a cluster or even across different computers connected by ethernet. Each process executes the same program, and its role is determined by its rank, for example in a setup where one rank distributes work and collects the results, while the remaining ranks do the computation.

The Mandelbrot set is a natural test case. It is embarrassingly parallel, as the only information needed for a process to do computation of a pixel is the location, in the complex plane, of the pixel and can therefore be computed independently by arbitrarily many workes. The Mandelbrot set is the set of points $p_0 = x + iy$ in the complex plane for which the sequence

$$p_{n+1} = p_n^2 + p_0 \tag{1}$$

remains bounded. Each pixel is mapped to a point (x, y), and the recursion is iterated until either $|p_n| > 2$, which guarantees the divergence of the sequence, or $n = n_{\max}$ for which it is assumed that the sequence remains bounded and the point is contained in the Mandelbrot set. The pixel value is kept at zero for points contained in the Mandelbrot set and is the number of iterations performed for points generating a divergent series, i.e. not contained in the Mandelbrot set. The interesting task is now to find out how to best distribute the work across processes. ***Processes*** are in MPI terms called ranks, and from here on out the notion of a process will in this report also be called a ***rank***.

A note on code snippets: Throughout the report we will include relevant code snippets. All code is written in python. Some snippets will be declared as pseudo-code and will simply show the relevant logic. Other code snippets may include a comment showing the source. In such snippets, some filler code such as timings and other data collecting will be removed to highlight the relevant logic and avoid bloat.

1.1 Domain decomposition

The cost of a pixel varies strongly. Points outside the Mandelbrot set typically escape the computation loop after a few iterations, whereas points inside the set always run the full $n_{\max}$ iterations. These expensive pixels are not spread uniformly but clustered in a region, and it is important to note that the total runtime of a parallel implementation is bounded by the runtime of the slowest rank. Any decomposition used, hands each rank a subset of the image, and thus gives far more work to a rank covering some regions inside the Mandelbrot set than other ranks covering other regions. Thus, a key method in this report is how to split the domain into many smaller blocks, so that each rank receives a mix of cheap and expensive regions.

Two easy ways of decomposing the image comes to mind. Dividing the image into subsections of k rows of the image or dividing the image into k rectangular regions in a grid structure. The first is by far the simplest. Therefore we do some comparisons of the options in the MPI framework. Namely whether or not the communication between ranks stops execution of the program and waits for the data to be completely transferred (***Blocking*** send/recv) or ranks instead continue on in the program script while the transfer of data is reading and writing to and from variables in the script (***Non-blocking*** send/recv).

2 Blocking vs. Non-blocking on row domain decomposition

We wish to compare using blocking and non-blocking implementations of a row-wise distribution in order to see if there is a difference between the performance for two in the standard Mandelbrot set with $x = [-2.2; 0.75]$ and $y = [-1.3; 1.3]$. We use 1000x1000 image size and a chunk size of 10.

We implement the blocking version naively (pseudo-code below)

```
comm.Barrier()
if rank == 0:
    comm.Recv(image[other_start:other_end, :], source=other_rank)
else:
    comm.Send(local_image, dest=0)
```

And the non-blocking version is implemented as below (pseudo-code below):

```
if rank == 0:
    reqs = []
    for other_rank in range(1, size_mpi):
        reqs.append(comm.Irecv(image[other_start:other_end, :], source=other_rank, tag =
    tag_img))
    MPI.Request.Waitall(reqs)
else:
    reqs = []
    reqs.append(comm.Isend(local_image, dest=0, tag = tag_img))
    MPI.Request.Waitall(reqs)
```

In fig. 1 we see the results of the performance benchmarking between the two implementations,

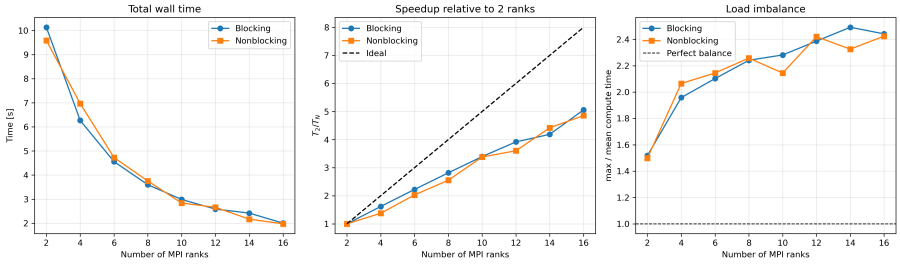


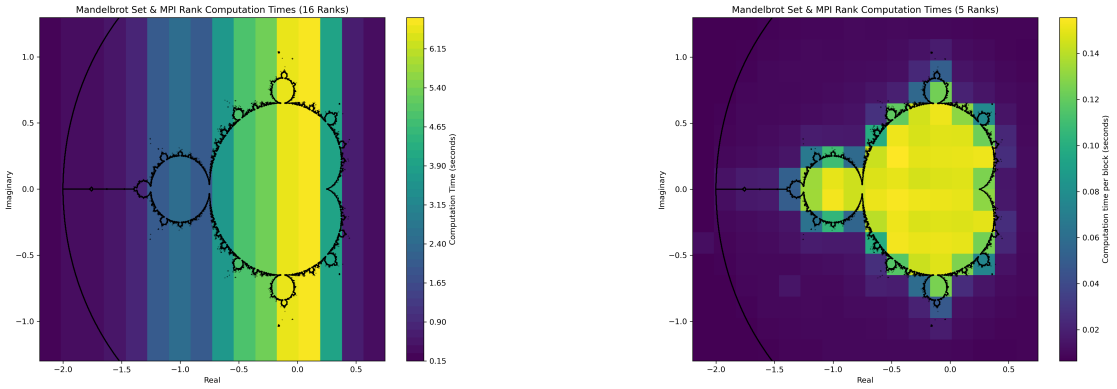
Figure 1: Scaling test between blocking and non-blocking implementations

We see that there is no major difference between the two implementations, this is expected as the image is so small that the transfer time and communication between ranks is very small compared to the computation loop which is the same between the two implementations. One could potentially see a

difference between the two implementations if one increased the image size, leading to a larger transfer time and letting communication play a bigger role in the total time spent.

3 Row vs Grid

Even though the rows actually distribute the work quite fairly. As most chunks contains at least some pixels that break out of the computation loop quickly and some pixels inside the Mandelbrot set. There exists a natural bottleneck in how small we are able to make the chunks using only row decomposition. We can only make the chunks as small as the width of the image. By going into a grid based approach where each chunk is a rectangular subsection of the image, we can make the chunks even smaller in size down to having only a single pixel in each chunk (illustrated in fig. 2). This is the strategy we will try to take a deeper dive into.



(a) Row distribution

(b) Grid distribution, 16×16 blocks

Figure 2: Plot shows time spent by each rank in their own section of the domain, visualized with the color intensity. Note that the scales are different, as maximum work-time the row implementation is a lot higher. Both plots show blocking implementations on a 1000×1000 grid. The left plot shows a blocking implementation with row-wise domain decomposition using 16 ranks. The plot on the right shows a grid decomposition using 256 blocks and 5 ranks (which is only 4 working ranks since rank zero only distributes).

Grid decomposition with static scheduling

To get started we utilize a static distribution of the regions in the complex plane. Figure 2b shows a static grid decomposition, where the domain is split into $n \times n$ blocks (n is provided by the user). Rank 0 does not compute anything, it builds a map of all blocks (index and a rectangular subsection of the complex plane given in the arguments) and sends each worker a set of consecutive block indices. The chunks/tasks are generated using

```
1       #src/Blocking/Mandelbrot_Grid.py
2       chunk_map = np.zeros((NUM_GRIDS, 5), dtype=int)
3
4       for idx in range(chunk_map.shape[0]):
```

```
        x_coord = idx // chunk_per_axis
        y_coord = idx % chunk_per_axis

        start_x = x_coord * n_x // chunk_per_axis
        end_x = (x_coord + 1) * n_x // chunk_per_axis

        start_y = y_coord * n_y // chunk_per_axis
        end_y = (y_coord + 1) * n_y // chunk_per_axis

        chunk_map[idx] = [idx, start_x, end_x, start_y, end_y]
```

If the number of blocks is not divisible by the number of workers, the remaining blocks are handed out one at a time, starting from rank 1. Since consecutive indices run along one column of the grid, each worker receives a contiguous vertical strip of the domain, so with 4 workers and 16×16 blocks this is effectively the same as a strip decomposition with 4 strips. The finer blocks reveal where the time is spent, but the load imbalance remains. Distributing the blocks cyclically or dynamically, as done in the following sections, addresses this.

Grid decomposition with dynamic scheduling

Dynamic scheduling is an obvious approach for combatting load imbalance. In particular for the grid decomposition, which provides a lot of flexibility and varying computation times in each grid. However, opting for dynamic scheduling, rather than a simpler load balancing scheme such as Round Robin, introduces a lot more overhead. This overhead, fortunately, also presents some opportunities to get creative with the communication.

By using the setup where rank 0 is the master rank in charge of distributing work and collecting results, we can borrow the decomposition from the static scheduled grid decomposition. Since, each task in the `chunk_map` consists of 5 integers we can skip some overhead be simply distributing the work using a blocking send. Given the small size of the tasks, we do not expect to see a significant performance loss from this, while it saves us from keep track of all send requests.

We do, however, need to consider how we want to receive the the completed tasks. Since, we will have `size_mpi-1` working ranks, we would be introducing a massive bottle neck by choosing a blocking receive. Hence, we setup a buffer for each working rank

```
    # src/Nonblocking/dynamic_scheduling.py

    # + 6 to accomdate some timing results from the rank
    recv_buffers = [
        np.zeros((nx_local * ny_local + 6), dtype=float) for _ in range(size_mpi)
    ]
    recv_reqs = [MPI.REQUEST_NULL] * size_mpi
```

This allows us to send 2 initial jobs to each worker and expose a recv request

```
    #src/Nonblocking/dynamic_scheduling.py
    for worker in range(1, size_mpi):
        for _ in range(2):
            if tasks_sent < NUM_GRIDS:
                task = chunk_map[tasks_sent]
                master_send_time -= perf_counter()
                comm.Send(
                    task, dest=worker, tag=0
```

```
                )  # Use blocking send (small package and the recieve is ready)
                master_send_time += perf_counter()
                worker_queue[worker].append(task)
                tasks_sent += 1

        # Post the first non-blocking recieve for this worker
        if len(worker_queue[worker]) > 0:
            recv_reqs[worker] = comm.Irecv(recv_buffers[worker], source=worker, tag=1)
```

The final important detail for the master rank, is to keep track of the number of completed tasks, and use `worker = MPI.Request.Waitany(recv_reqs)`, to listen for completed tasks. Every completed task allows the master rank to add the completed grid to the full image, send the next task and post a new non-blocking receive to the worker. Once the number of completed tasks, equals the length of `chunk_map`, a shutdown signal is sent to the workers.

On the working ranks, we want to allow for simultaneous communication and computations. This limits us to purely non-blocking send/recv. This is achieved by allocating 3 buffers for computation and communication. While 2 buffers are sufficient to allow for simultaneous communication and computation, we do need to ensure the buffer is ready for reuse, once we get to the buffer a second time. Hence, adding the 3rd buffer, reduces the chance of having to wait for the buffer to get ready. Hence we initialize a working rank by

```
#src/Nonblocking/dynamic_scheduling.py
# Prepare buffers
NUM_BUFFERS = 3

# Create three dimensions (each is an image)
image_buffer = np.zeros((NUM_BUFFERS, nx_local, ny_local), dtype=float)
info_buffer = np.zeros((NUM_BUFFERS, 5), dtype=int)

# Initialize requests
send_reqs = [MPI.REQUEST_NULL] * NUM_BUFFERS
recv_reqs = [MPI.REQUEST_NULL] * NUM_BUFFERS

# Set the rank buffer index to shift between buffers
buffer_idx = 0

# Prepost the buffers
for i in range(NUM_BUFFERS):
    recv_reqs[i] = comm.Irecv(info_buffer[i], source=0, tag=0)
```

Now the working rank simply have to repeatedly run through the buffers when doing the computations and communication

```
#src/Nonblocking/dynamic_scheduling.py
while True:
    # Wait for the info_buffer to be ready
    recv_reqs[buffer_idx].Wait()

    # Load the necessary info for computation
    glb_idx, start_x, end_x, start_y, end_y = info_buffer[buffer_idx]

    # Catch the shutdown signal
```

```
        if glb_idx < 0:
            break

        # Ensure previous use of this image buffer is done
        send_reqs[buffer_idx].Wait()

        # Post next recieve request, so it can be done while computing
        recv_reqs[buffer_idx] = comm.Irecv(info_buffer[buffer_idx], source=0, tag=0)

        # Setup pointer directly into the current slice of the image buffer
        current_image = image_buffer[buffer_idx]
        current_image.fill(0)

        # ...
        ## <Mandlebrot computation and timing>
        # ...

        # Bundle data and send the image back to rank 0
        data_bundle = np.append(current_image.flatten(), time_package)
        send_reqs[buffer_idx] = comm.Isend(data_bundle, dest=0, tag=1)

        # Move buffer to next slice
        buffer_idx = (buffer_idx + 1) % NUM_BUFFERS
```

Before implementing the grid decomposition, the setup was tested using dummy data. The dummy data was 50 grids of 100×100 points. On rank 1, we artificially slowed the computation by inserting a `sleep(0.1)`-command. Thus, if everything ran as expected we would see rank 1 computed significantly less tasks than the other two ranks. Once, the scheduling acted as expected the output was:

```
--- SCHEDULING RESULTS ---
Worker 1 completed 7 tasks.
Worker 2 completed 21 tasks.
Worker 3 completed 22 tasks.

    Master send time = 0.0002
    Master wait time = 0.9732
    working idle time = 0.0351
    Computation time = 1.8207
```

Performance comparison for scaling images

This section features a comparison between the performance of static row decomposition with blocking communication (Row S/B), static grid decomposition with blocking communication (Grid S/B), and the dynamic grid decomposition with non-blocking communication described in the section above (Grid D/N). For both grid decompositions all sub-grids are 256×256. All computations have been carried out on XeonE5_2650v4 CPUs. These CPUs have 24 cores, and most tests are carried out such that we use a a multiple of 24 processes. Thus, most results are generated by completely taking up each CPU. The largest tests use 72 processes. This is due to restrictions on the available "hpcintro"-queue, and to avoid long queue times on "hpc".

Results visualized in Figure 3 shows the time breakdown for each rank for smallest and largest image tested. Overall the implementations behave as expected. Both static scheduling implementations are

challenged with severely unbalanced work loads, and long idle times for higher ranks. The high idle times, are caused by the receiving loop at rank 0, where ranks are contacted in ascending order. The dynamic scheduling is way more balanced and the idle time has been eliminated. For the smaller image of 5000×5000, we do still hit some of the wait-statements, where we ensure previous use of the buffer is done (ie the Isend is done). This time is fairly constant, and comes to around 1 second regardless of image size. Hence, for $5k \times 5k$, this contributes about a 5'th of the total workload, while it is negligible at larger scale.

When looking at fig. 3 (static, left and middle) we see that the rank time histogram outlines the Mandelbrot set, this is because both static implementations decompose the problem around its symmetries. As a result we see how the ranks who compute the set around $x = 0$, are tasked with a significantly heavier computational section than the boundary workers. The fact that both the row and static grid implementation perform so similarly is again a consequence of our naive static scheduling of the grid decomposition, where each worker get consecutive grids from the same row. Hence, at lower resolution the Grid S/B, implementation is simply the row decomposition with more communication overhead. At scale both decompositions, suffers from the computational load being concentrated in a few rows. For Row S/B this means that the load distribution is shifted right, with a large spike on the ranks around 50. The grid distribution remains more dispersed, but have multiple smaller spikes.

Using dynamic scheduling (right in fig. 3) we concentrating the computational cost in a few workers. This results in a speed-moving from around 2x towards 3x at larger scales. This is further investigated in fig 4.

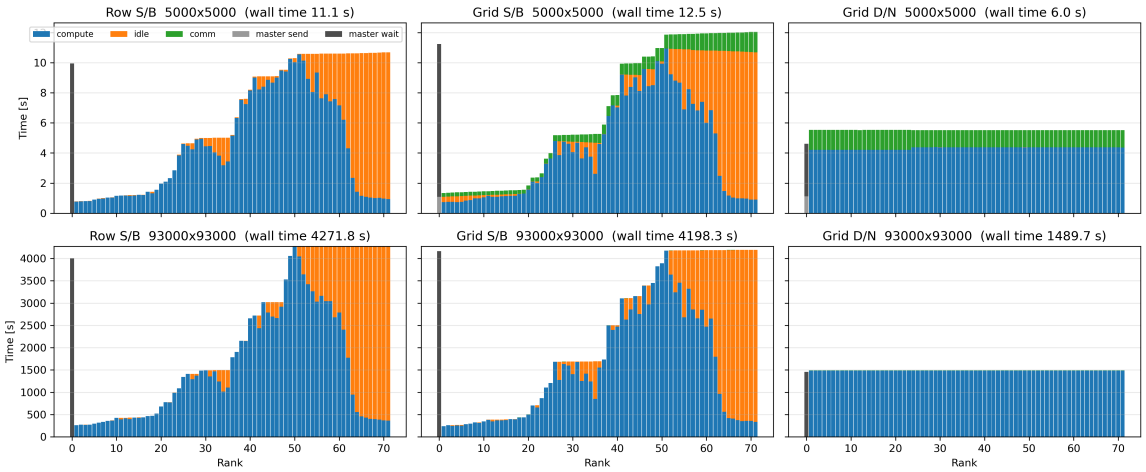


Figure 3: Time breakdown per rank for Row and Grid decompositions using 72 ranks (1 master + 71 workers). "S/B" denotes static scheduling and blocking communication. "D/N" denotes dynamic scheduling and non-blocking communication. For each worker, the bars show time spent computing (blue), idle (orange) and communicating (green). Rank 0 is the master, which performs no computation. Thus its bars show time spent sending work (light grey) and waiting for results (dark grey). The height of each bar corresponds to when that rank finished.

Figure 4 compares the scaling performance of the 3 implemtations. The left plot shows that the wall time of all implementations scales linearly with the number of pixels, with dynamic scheduling consistently faster. The main causes have been discussed looking at fig 3, but they are re-emphasised in the middle plot, which shows the ratio between the maximum and the mean compute time across the workers. With static scheduling, each worker is assigned a fixed region of the complex plane, and

workers whose regions cover the interior of the Mandelbrot set must perform the maximum number of iterations for every pixel. As a result, the slowest worker spends around 2.8 times longer computing than the average worker, independent of image size, since the imbalance is determined by the geometry of the set rather than the resolution. With dynamic scheduling the ratio is around 1.0, as workers request new tiles as soon as they finish, so the work is evenly distributed. The right plot quantifies the resulting speedup. At low resolution we find a 1.8 and 2.1 speedup. At the higher resolutions we find around 2.8 for both. The speedup exceeds the static imbalance at larger sizes because the static implementation additionally collects results from the workers in rank order, while at 5000x5000 the gain is smaller since fixed overheads make up a larger fraction of the shorter runtime.

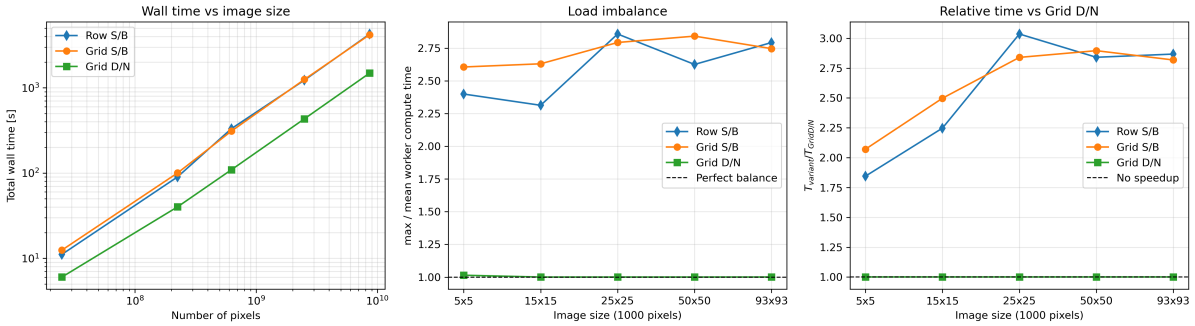


Figure 4: Comparison of Row and grid decompositions with static (S) and Dynamic (D) scheduling with blocking (B) and non-blocking (N) communication. Each run uses 72 ranks. Leftmost plot is the total wall time as a function of the number of pixels. Middle plot shows load imbalance, measured as the ratio between the maximum and the mean compute time across the workers, where 1 indicates perfect balance. Rightmost plot shows speedup of dynamic over static scheduling, T-static / T-dynamic, for the image sizes where both variants were run.

Performance benchmark using 1 node vs. multiple nodes.

We want to see how the performance looks across multiple nodes, to see if there is any overhead visually impacting our performance. This overhead would come from communication between our nodes, however, looking at fig. 5 we see that there is not an overhead that degrades our performance measurable. We see that, as expected the dynamic grid is performing much faster than the other two implementations, we see this is also reflected in the load balancing, where the dynamic grid is nearly perfectly balanced.

If there was any significant overhead between nodes, the lines would drop and level off as the worker count increase, we may see some of that effect for the static rows, but there is no way to tell with only 3 nodes, with more nodes available one could potentially see that the effects of communication between nodes. But for our tests it is not observed and therefore we cannot say for certain if it has impact or not. What we can see, is that, at this scale there is not any significant overhead.

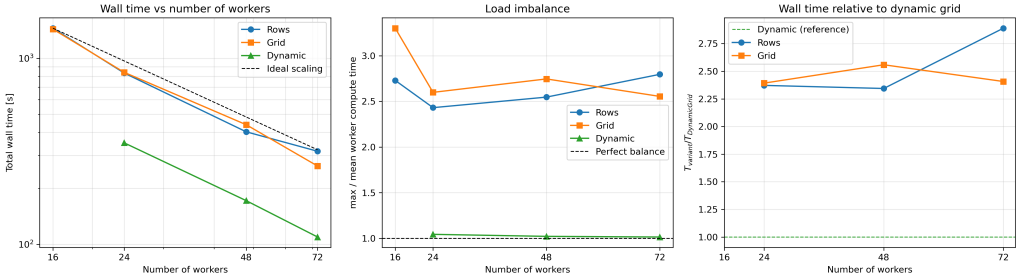


Figure 5: Performance benchmarks for increasing number of workers using 25000x25000 pixel image. Left: Wall time vs. number of workers for S/R (static rows), S/G (static grid) and D/G (dynamic grid). Middle: Load imbalance between S/R, S/G and D/G, we observe that the dynamic grid has almost perfect balancing. Right: We see that the static grid is closer in speedup to the dynamic grid than the rows

We can say that the implementation with dynamic grid scales quite good with added number of workers, meaning, if you use more resources to comppute the problem using this implementation, the resources are utilized well.

Conclusion

In this report we parallelized the computation of the Mandelbrot set with MPI and investigated how the work should be distributed. What we found quickly is that the cost of a pixel varies wildly across the domain, so total runtime is dictated by the slowest rank rather than by the total amount of work.

This is supported by our results. Blocking and non-blocking communication performed the same for row decomposition on a 1000×1000 image as the computation dominates the communication at that size. Moving to a grid decomposition did not help in itself since our naive static scheduling gives each worker consecutive blocks, which effectively gives a row decomposition. Only when introducing dynamic scheduling did the load imbalance disappear. The slowest worker went from spending around 2.8 times the average compute time to around 1.0, which gave a speedup of 2.1-2.8 over the static implementations. The non-blocking communication made this practical, as it keeps the added scheduling overhead small.

Finally, the dynamic grid scaled well with the number of workers, and we did not see any significant overhead when using multiple nodes. With only three nodes available we cannot say whether this changes at larger scale.